

Laboratório — Funções e procedimentos em PL/pgSQL no iximiuz Labs

Objetivo

Criar, chamar, corrigir e inspecionar rotinas armazenadas em um PostgreSQL executado em contêiner, usando o mesmo ambiente da Aula 04.

Antes de começar

- Use um computador do laboratório, preferencialmente em dupla.
- Abra <https://labs.iximiuz.com/playgrounds/BD2-AULA05-dedf2f1b> e clique em **Start Playground**. A máquina costuma subir em menos de um minuto. É o ambiente preparado para esta aula: o PostgreSQL 17 já vem baixado e os scripts do laboratório já estão em ~/lab05.
- O playground abre no navegador com duas abas úteis: **Term** (terminal) e **IDE** (editor).
- Para colar um comando no terminal do navegador, use **Ctrl + Shift + V** ou o botão direito do mouse.
- A sessão dura algumas horas. Ao encerrar, a plataforma oferece **Stop**, que guarda o estado do ambiente para você retomar depois, e **Destroy**, que apaga tudo.
- Mesmo guardando o ambiente, anote os comandos e os resultados que interessam: é o que você vai reler para estudar.
- Se estiver somente no celular, acompanhe cada comando e anote o resultado apresentado em aula.

Parte 1 — Subir o PostgreSQL

```
podman run -d --name bd2-pg \  
-e POSTGRES_PASSWORD=bd2 \  
-e POSTGRES_DB=loja \  
docker.io/library/postgres:17
```

Na primeira execução o Podman baixa a imagem; isso leva alguns instantes.

A senha é apenas didática e serve para um ambiente de estudo. Nunca reutilize senhas de exemplo em sistemas reais.

Confira se o servidor já aceita conexões:

```
podman ps  
podman exec bd2-pg pg_isready -U postgres
```

Resultado esperado: o contêiner aparece na lista e o pg_isready responde accepting connections.

Parte 2 — Entrar no psql

```
podman exec -it bd2-pg psql -U postgres -d loja
```

Resultado esperado: o prompt muda para loja=#. A partir daqui, tudo o que for digitado é SQL.

Comandos úteis do psql:

Comando	Efeito
\dt	lista as tabelas
\df	lista as funções e os procedimentos
\sf nome	mostra o código-fonte de uma rotina
\q	sai do psql

Parte 3 — Criar o esquema da loja

O arquivo 01_esquema.sql já está na máquina, em ~/lab05. Ele cria quatro tabelas e insere os dados de exemplo. Saia do psql com \q e execute:

```
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab05/01_esquema.sql
```

Resultado esperado: uma sequência de DROP TABLE, CREATE TABLE e INSERT sem nenhuma linha ERROR.

Volte para o psql:

```
podman exec -it bd2-pg psql -U postgres -d loja
```

O conteúdo do arquivo está reproduzido abaixo para você acompanhar pelo celular e entender o que foi criado:

```

CREATE TABLE cliente (
    id      serial PRIMARY KEY,
    nome    text NOT NULL,
    cidade text NOT NULL
);

CREATE TABLE produto (
    id      serial PRIMARY KEY,
    nome    text NOT NULL,
    preco   numeric(10,2) NOT NULL CHECK (preco > 0),
    estoque integer NOT NULL DEFAULT 0 CHECK (estoque >= 0)
);

CREATE TABLE pedido (
    id          serial PRIMARY KEY,
    cliente_id  integer NOT NULL REFERENCES cliente (id),
    data_pedido date NOT NULL DEFAULT CURRENT_DATE,
    situacao    text NOT NULL DEFAULT 'aberto'
);

CREATE TABLE item_pedido (
    pedido_id    integer NOT NULL REFERENCES pedido (id),
    produto_id   integer NOT NULL REFERENCES produto (id),
    quantidade   integer NOT NULL CHECK (quantidade > 0),
    preco_unitario numeric(10,2) NOT NULL CHECK (preco_unitario > 0),
    PRIMARY KEY (pedido_id, produto_id)
);

INSERT INTO cliente (nome, cidade) VALUES
    ('Ana Souza', 'Palmas'),
    ('Bruno Lima', 'Porto Nacional'),
    ('Carla Dias', 'Paraíso do Tocantins');

INSERT INTO produto (nome, preco, estoque) VALUES
    ('Teclado', 120.00, 10),
    ('Mouse', 80.00, 15),
    ('Monitor', 900.00, 4),
    ('Cabo HDMI', 35.00, 25);

INSERT INTO pedido (cliente_id) VALUES (1), (1), (2);

INSERT INTO item_pedido (pedido_id, produto_id, quantidade, preco_unitario)
VALUES
    (1, 1, 2, 120.00),
    (1, 4, 1, 35.00),
    (2, 3, 1, 900.00),
    (3, 2, 3, 80.00);

```

Confira:

```
\dt
SELECT * FROM produto ORDER BY id;
```

Onde ficam os arquivos: ~/lab05 guarda 01_esquema.sql (o que você acabou de rodar) e 02_rotinas.sql (as rotinas prontas). Abra os dois pela aba **IDE** para ler o código com cores e comentários.

Use 02_rotinas.sql apenas para **conferir** cada rotina *depois* de escrevê-la você mesmo. Rodar o arquivo inteiro no começo cria tudo de uma vez e você perde justamente o que a aula treina.

Parte 4 — O problema do dia

Quanto custa o pedido 1?

```
SELECT SUM(quantidade * preco_unitario) AS total
FROM item_pedido
WHERE pedido_id = 1;
```

Resultado esperado: 275.00.

A consulta funciona, mas precisa ser reescrita em todo lugar que precisar do total: no site, no aplicativo e no relatório. Vamos guardá-la dentro do banco.

Parte 5 — Primeira função (LANGUAGE sql)

```
CREATE OR REPLACE FUNCTION fn_desconto_sql(
    p_preco      numeric,
    p_percentual numeric
)
RETURNS numeric
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT round(p_preco - p_preco * p_percentual / 100, 2);
$$;
```

Chame a função:

```
SELECT fn_desconto_sql(120.00, 10);
SELECT nome, preco, fn_desconto_sql(preco, 15) AS promocao FROM produto ORDER BY id;
```

Resultado esperado: 108.00 na primeira consulta e uma coluna de preços promocionais na segunda.

Parte 6 — A mesma regra em PL/pgSQL

```
CREATE OR REPLACE FUNCTION fn_desconto(  
    p_preco      numeric,  
    p_percentual numeric DEFAULT 10  
)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_valor numeric;  
BEGIN  
    v_valor := p_preco - p_preco * p_percentual / 100;  
    RETURN round(v_valor, 2);  
END;  
$$;
```

```
SELECT fn_desconto(120.00, 25);  
SELECT fn_desconto(120.00);
```

Resultado esperado: 90.00 e 108.00. Na segunda chamada, o parâmetro omitido assumiu o valor-padrão 10.

Parte 7 — Função que consulta o banco

```
CREATE OR REPLACE FUNCTION fn_total_pedido(  
    p_pedido_id integer  
)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_total numeric;  
BEGIN  
    SELECT COALESCE(SUM(quantidade * preco_unitario), 0)  
        INTO v_total  
        FROM item_pedido  
        WHERE pedido_id = p_pedido_id;  
  
    RETURN v_total;  
END;  
$$;
```

```
SELECT fn_total_pedido(1);  
SELECT id, fn_total_pedido(id) AS total FROM pedido ORDER BY id;
```

Resultado esperado: 275.00, 900.00 e 240.00.

Parte 8 — Função que devolve uma tabela

```
CREATE OR REPLACE FUNCTION fn_pedidos_do_cliente(  
    p_cliente_id integer  
)  
RETURNS TABLE (  
    pedido_id integer,  
    data_pedido date,  
    valor_total numeric  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RETURN QUERY  
        SELECT p.id, p.data_pedido, fn_total_pedido(p.id)  
            FROM pedido p  
           WHERE p.cliente_id = p_cliente_id  
           ORDER BY p.id;  
END;  
$$;
```

```
SELECT * FROM fn_pedidos_do_cliente(1);
```

Resultado esperado: duas linhas, com os totais 275.00 e 900.00.

Atenção: os nomes declarados em RETURNS TABLE viram variáveis dentro da função. Por isso o SELECT usa o apelido p. em todas as colunas. Sem essa qualificação, o PostgreSQL responde column reference "data_pedido" is ambiguous.

Parte 9 — Procedimento

```
CREATE OR REPLACE PROCEDURE sp_baixar_estoque(
    p_produto_id integer,
    p_quantidade integer
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_estoque integer;
BEGIN
    SELECT estoque
    INTO v_estoque
    FROM produto
    WHERE id = p_produto_id;

    IF v_estoque IS NULL THEN
        RAISE EXCEPTION 'Produto % não encontrado.', p_produto_id;
    END IF;

    IF v_estoque < p_quantidade THEN
        RAISE EXCEPTION 'Estoque insuficiente: há % unidades e foram pedidas %.',
            v_estoque, p_quantidade;
    END IF;

    UPDATE produto
    SET estoque = estoque - p_quantidade
    WHERE id = p_produto_id;

    COMMIT;
END;
$$;
```

Chame com CALL e confira o efeito:

```
SELECT id, nome, estoque FROM produto WHERE id = 3;
CALL sp_baixar_estoque(3, 1);
SELECT id, nome, estoque FROM produto WHERE id = 3;
```

Resultado esperado: o estoque do monitor cai de 4 para 3.

Parte 10 — Provocar um erro de propósito

```
CALL sp_baixar_estoque(3, 500);
```

Resultado esperado: ERROR: Estoque insuficiente: há 3 unidades e foram pedidas 500.

Confirme que nada foi alterado:

```
SELECT id, nome, estoque FROM produto WHERE id = 3;
```

A rotina interrompeu a execução antes do UPDATE. A regra ficou dentro do banco e vale para qualquer aplicação que se conecte a ele.

Parte 11 — Inspecionar e manter as rotinas

```
\df
\sfn fn_total_pedido
```

Uma consulta equivalente ao \df, útil fora do psql:

```
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'public'
ORDER BY routine_name;
```

Para remover uma rotina, informe também os tipos dos parâmetros:

```
DROP FUNCTION fn_desconto_sql(numeric, numeric);
\df
```

Parte 12 — Desafio (opcional)

Escreva um procedimento que abra um pedido e devolva o identificador gerado:

```
CREATE OR REPLACE PROCEDURE sp_abrir_pedido(
    p_cliente_id integer,
    INOUT p_pedido_id integer
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO pedido (cliente_id)
    VALUES (p_cliente_id)
    RETURNING id INTO p_pedido_id;
END;
$$;

CALL sp_abrir_pedido(2, NULL);
```

Resultado esperado: o CALL devolve o número do pedido recém-criado.

Depois, responda: por que esse procedimento não poderia ser escrito como uma função que também faz COMMIT?

Checklist de conclusão

- ☐ Subi o PostgreSQL em contêiner e entrei no psql.
- ☐ Criei o esquema da loja e conferi os dados.
- ☐ Criei uma função em LANGUAGE sql e outra em LANGUAGE plpgsql.
- ☐ Usei parâmetro com valor-padrão.
- ☐ Criei uma função que devolve um valor e outra que devolve uma tabela.
- ☐ Criei um procedimento e o executei com CALL.
- ☐ Li uma mensagem de erro e expliquei o que ela significa.
- ☐ Listei as rotinas com \df e removi uma delas.

Encerramento

Saia do psql:

```
\q
```

Se quiser apenas desligar o banco, mantendo o playground aberto:

```
podman stop bd2-pg
```

Ao fechar o playground, escolha com atenção:

Opção	Efeito
Stop	guarda o estado do ambiente; você pode retomá-lo depois, com o contêiner e os arquivos como estavam
Destroy	apaga o ambiente definitivamente

Para continuar o estudo em casa, use **Stop**.